Základy SQL a práce s daty

Jiří Mareš

DATA academy



Co nás čeká

Spousta práce!

Data versus databáze

Co jsou to data?

Data jsou v informatice údaje zaznamenané v digitální (číselné) podobě určené k počítačovému zpracování.

Pojem data je často libovolně zaměňován s pojmem informace, nicméně v rámci teorie informace formulované kybernetikem Norbertem Wienerem jsou data prostým záznamem hodnot, a informace se z nich stávají až po jejich výkladu v kontextu s využitím znalostí.

Co je to databáze?

Databáze (neboli datová základna, též databanka) je systém souborů s pevnou strukturou záznamů. Tyto soubory jsou mezi sebou navzájem propojeny pomocí klíčů. V širším smyslu jsou součástí databáze i softwarové prostředky, které umožňují manipulaci s uloženými daty a přístup k nim.

Práce s daty

Jak s daty pracujeme?

Filtrujeme / vyhledáváme / analyzujeme

Agregujeme (=slučujeme)

Řadíme

Obohacujeme / spojujeme

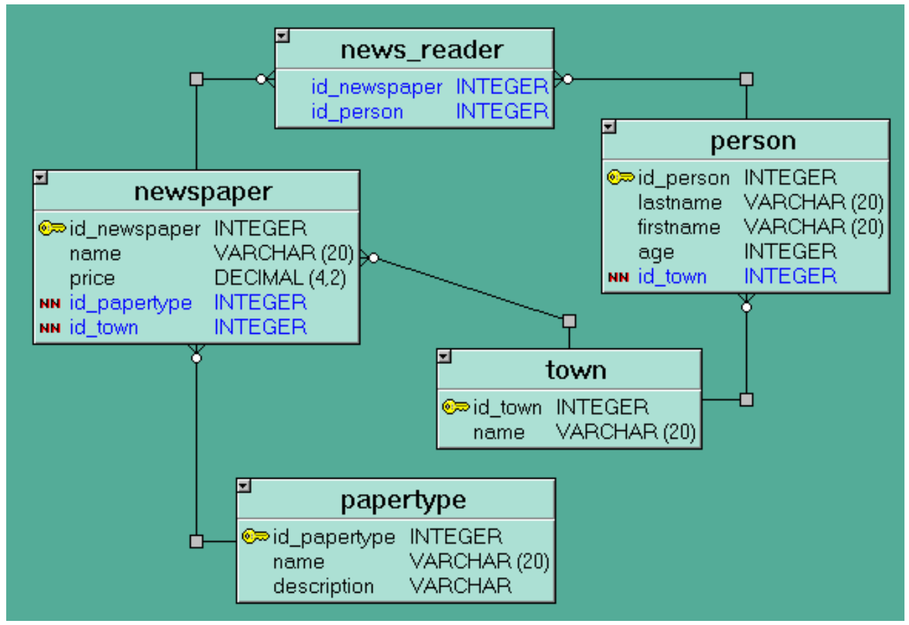
Vizualizujeme

Relační databáze

Co to je?

Relační databáze je založena na tabulkách, jejichž řádky obvykle chápeme jako záznamy a eventuálně některé sloupce v nich (tzv. cizí klíče) chápeme tak, že uchovávají informace o relacích mezi jednotlivými záznamy v matematickém slova smyslu.

Termín relační databáze definoval Edgar F. Codd v roce 1970.



DWH – Data Warehouse

Primární klíč – jednoznačný identifikátor záznamu v každé tabulce.

Cizí klíč – odkaz na primární klíč jiné tabulky.

V DWH všechny tyto atributy končí na KEY – každý sloupec s tímto názvem je tak buď primární klíč nebo cizí klíč. Název je až na výjimky tvořen jako *<prefix tabulky>_KEY.*

Příklad:

Tabulka ACCOUNT_STATUS má prefix ACCST, její primární klíč má tak název ACCST_KEY a veškeré atributy (pokud nejsou cizími klíči) začínají tímto prefixem.

Upozornění:
Primární klíč může být tvořen i více sloupci!

DWH – Data Warehouse

Unikátní klíč – většina tabulek v DWH má kromě technického primárního klíče i (významový) unikátní klíč, který odpovídá identifikátoru ze zdroje. Nazývá se <prefix tabulky>_SOURCE_ID a bývá doplněn o identifikátor zdrojového systému <prefix tabulky>_SOURCE_SYSTEM_ID.

SRCSYS_ID	SRCAPPL_ID	SRCSYS_NAME	SRCSYS_SHORT_NAME
SB1	SB	STARBANK LOANS	SB LOANS
SB10	SB	SBD VIRTUAL DEALS 0	SB
SB2	SB	STARBANK COLLATERALS	SB COL
SB3	SB	STARBANK ZPS	SB ZPS
SC0	SC	SC OFFICE	SCO
SC1	SC	SC OFFICE (DERIVATIVES)	SC1
SC2	SC	CTS CONTRACTS	SC2
SC3	CO	SC OFFICE (OPTIONS)	SC3
SC9	SC	SC OFFICE	SC9
SE0	SE	SEE	SEE
SL0	SL	SPECIALIZED LENDING	SL
SMC0	SMC	SMARTCASE	SMARTCASE
SS0	SAS	SAS	SAS
ST0	ST	CONSTRUCTION SAVINGS DEPOSITS	CSST
ST1	ST	CONSTRUCTION SAVINGS LOANS	CSST

DWH – Data Warehouse

Auditní atributy – všechny opět začínají *<prefix tabulky>_ ; pro zpřehlednění budou názvy ve výčtu bez prefixu.*

DELETED_FLAG – příznak, jestli se jedná o smazaný záznam. Nabývá hodnot Y (ano), N (ne) a X (u technických záznamů).

INSERTED_DATETIME – datum a čas vložení záznamu (systémový čas)

UPDATED_DATETIME – datum a čas změny záznamu (systémový čas)

INSERT_PROCESS_KEY – číslo procesu, který záznam založil

UPDATE_PROCESS_KEY – číslo procesu, který záznam upravil

DWH – Data Warehouse

V DWH jsou některé tabulky historizované, to znamená, že je možné vidět, jak se jednotlivá data v čase měnila. Změny jsou zaznamenány na denní bázi, tedy pokud by u záznamu během jednoho dne došlo k několika změnám, pak bude zachycen stav po poslední změně.

Historické atributy – všechny opět začínají <*prefix tabulky*>_ *; pro zpřehlednění budou názvy ve výčtu bez prefixu.*

VALID_FROM – začátek platnosti záznamu

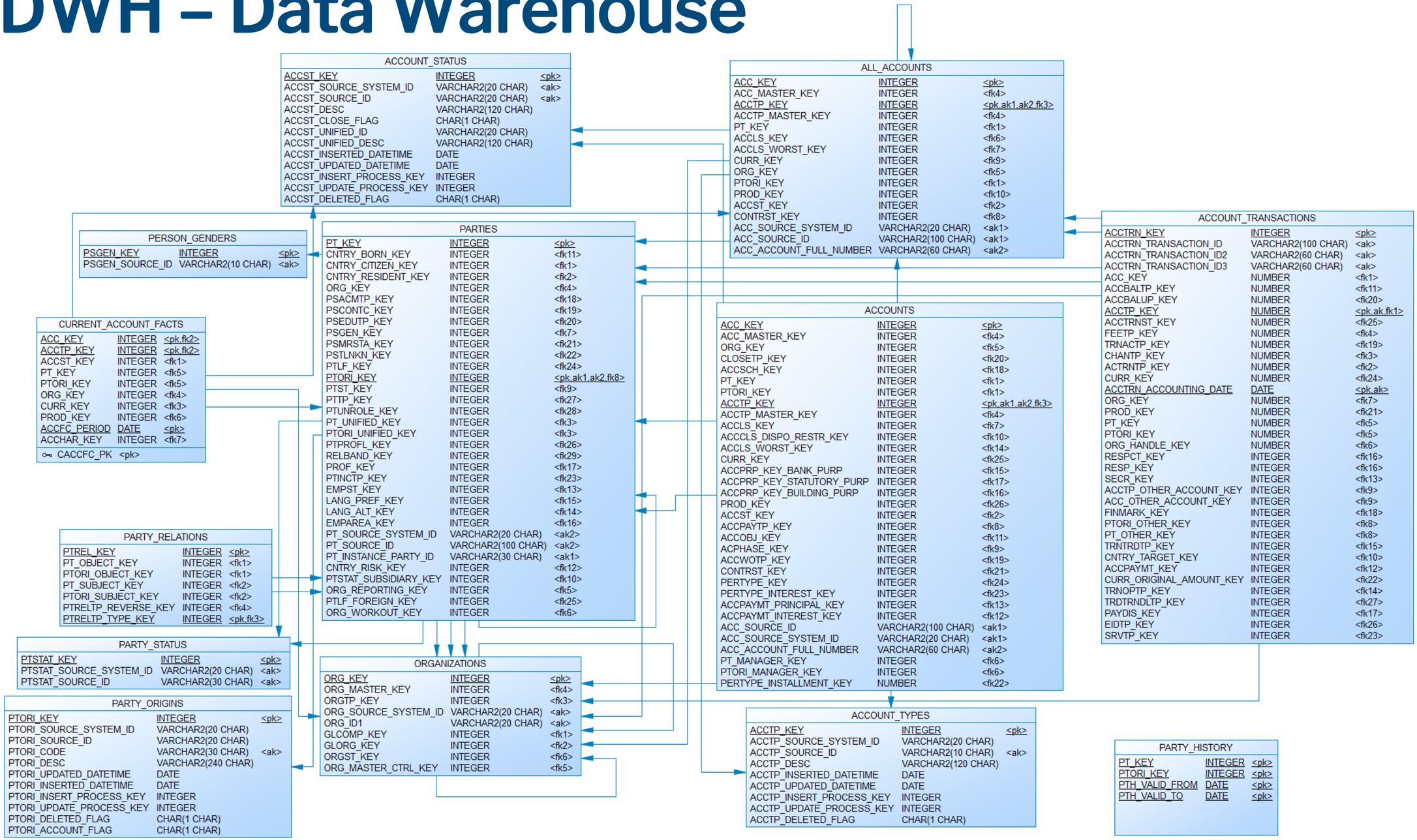
VALID_TO – konec platnosti záznamu; poslední (aktuální záznam) má platnost do 1.1.3000

CURRENT_FLAG – příznak, jestli se jedná o aktuální záznam; v praxi se příliš nepoužívá, preferuje se vždy časový řez pomocí platností.

Poznámka:

Atribut VALID_FROM není plněn systémovým časem, ale má „business“ význam. Odpovídá datu, ke kterému nám daný zdrojový systém poslal data. Obecně dnes zpracováváme data za včerejšek (po půlnoční uzávěrce), a tak i datum „včera“ se objeví ve VALID_FROM změněného (ale i nového) záznamu.

DWH – Data Warehouse



SQL

Co to je?

Structured Query Language - standardizovaný strukturovaný dotazovací jazyk, který je používán pro práci s daty v relačních databázích.

DDL

Data definition language operace umožňující vytvářet, upravovat a mazat objekty databáze.

Příklad:

CREATE <object>, ALTER <object>, DROP <object>, TRUNCATE, ...

DML

Data manipulation language - operace umožňující získávat, ukládat a mazat data v databázi.

Příklad:

SELECT, INSERT, UPDATE, DELETE, MERGE, ...

Motivační příklad

Kolik nových klientů za Starbanku bylo založeno nebo zrušeno v lednu 2026 a jaký je přírůstek / úbytek v procentech oproti stavu k ultimo prosince 2025.

SELECT

SELECT - vybírá data z databáze, umožňuje výběr podmnožiny a řazení dat.

Základní syntaxe:

```
SELECT
  <seznam sloupců>
FROM
  <schéma>.<tabulka>
;
```

Poznámka:
Místo seznamu sloupců můžete použít *, která značí všechny sloupce. Jednotlivé sloupce v <seznamu sloupců> jsou odděleny čárkou.

Úkol:

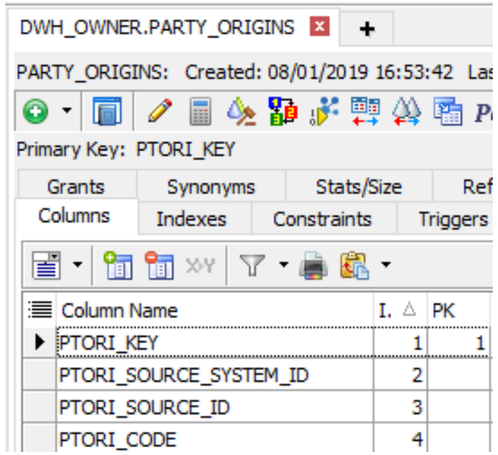
Jak poznáme klienty Starbank?

Připojte se do databáze DWHTS1 pod svým CENem a pomocí příkazu SELECT se podívejte na tabulku DWH_OWNER.PARTY_ORIGINS.

SELECT

Řešení:

*select * from dwh_owner.party_origins;*



PTORI_KEY	PTORI_SOURCE_SYSTEM_ID	PTORI_SOURCE_ID	PTORI_CODE	PTORI_DESC	PTORI_UPDATED_DATETIME	PTORI_INSERTED_DATETIME	PTORI_INSERT_PROCESS_KEY	PTORI_UPDATE_PROCESS_KEY	PTORI_DELETED_FLAG	PTORI_ACCOUNT_FLAG
3	EX0	PWC	PWC	KREDITNÍ KARTY	14/05/2011 19:39:59	04/04/2003 17:54:59	2	1671998	N	Y
6	EX0	IUK	IUK	INSTANČNÍ UNIFIKOVANÝ KLIENT	04/02/2012 11:12:11	04/04/2003 17:54:59	2	1896256	N	Y
8	EX0	DWH	DWH	CS DATAWAREHOUSE	14/05/2011 19:39:59	04/04/2003 17:54:59	2	1671998	N	N
5	EX0	EIGER	EIGER	DEBETNÍ A KREDITNÍ KARTY EIGER	14/05/2011 19:39:59	04/04/2003 17:54:59	2	1671998	N	N
7	EX0	ODS	ODS	CS OPERATIONAL DATASTORE	14/05/2011 19:39:59	04/04/2003 17:54:59	2	1671998	N	N
2	EX0	SB	SB	STARBANK	14/05/2011 19:39:59	04/04/2003 17:54:59	2	1671998	N	Y
1	EX0	SU	SU	SPOROŽIRO	14/05/2011 19:39:59	04/04/2003 17:54:59	2	1671998	N	Y
4	EX0	CRM	CRM	CRM	14/05/2011 19:39:59	04/04/2003 17:54:59	2	1671998	N	N
25	EX0	RS	RS	RISK SERVER	14/05/2011 19:39:59	19/02/2005 12:26:02	18885	1671998	N	N
24	EX0	SPUC	SPUC	SPRÁVCI ÚČTŮ	14/05/2011 19:39:59	19/02/2005 12:26:02	18885	1671998	N	N
20	EX0	BESCPI	BESCPI	BEHAVIORAL SCORING PI	14/05/2011 19:39:59	10/11/2003 15:02:49	1680	1671998	N	N
18	EX0	CIC	CIC	CIC	14/05/2011 19:39:59	10/11/2003 15:02:49	1680	1671998	N	N
21	EX0	CIS	CIS	CIS	14/05/2011 19:39:59	10/11/2003 15:02:49	1680	1671998	N	N
16	EX0	CSST	CSST	STAVEBNÍ SPOŘITELNA ČESKÉ SPOŘITELNY	14/05/2011 19:39:59	10/11/2003 15:02:49	1680	1671998	N	Y

Starbankový klient se pozná podle hodnoty **PTORI_KEY = 2**.

SELECT

Výsledná data nejsou nijak setříděná, řazená jsou (pseudo)náhodně. To můžeme napravit pomocí klauzule **ORDER BY**.

Základní syntaxe:

```
SELECT
  <seznam sloupců>
FROM
  <schéma>.<tabulka>
ORDER BY
  <seznam sloupců ASC / DESC>
;
```

Poznámka:
Místo seznamu sloupců můžete použít číslice, které odpovídají pořadí vybíraného sloupce.

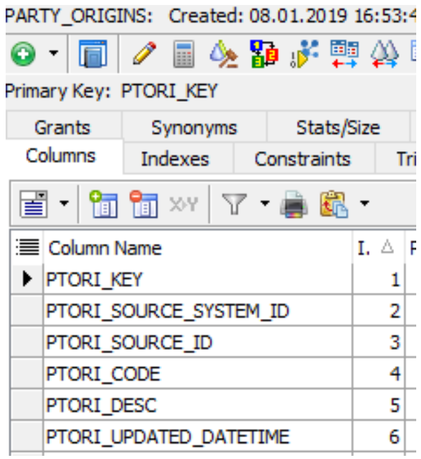
Úkol:

Setřiďte data podle popisku.

SELECT

Řešení:

*select * from dwh_owner.party_origins order by ptori_desc;*
*select * from dwh_owner.party_origins order by ptori_desc asc;*
*select * from dwh_owner.party_origins order by 5;*
*select * from dwh_owner.party_origins order by 5 asc;*



PTORI_KEY	PTORI_SOURCE_SYSTEM_ID	PTORI_SOURCE_ID	PTORI_CODE	PTORI_DESC	PTORI_UPDATED_DATETIME	PTORI_INSERTED_DATETIME	PTORI_INSERT_PROCESS_KEY	PTORI_UPDATE_PROCESS_KEY	PTORI_DELETED_FLAG	PTORI_ACCOUNT_FLAG
46	EX0	AMT	AMT	ASSET MANAGEMENT TOOL	18.01.2014 1:41:10	20.07.2013 1:22:06	2262431	2313977	N	Y
71	EX0	AMTUS	AMTUS	ASSET MANAGEMENT TOOL - USERS	20.12.2019 12:21:03	20.12.2019 12:21:03	3276598	3276598	N	N
70	EX0	ABS	ABS	ASSET-BACKED SECURITIES SUITE	13.04.2019 2:13:49	13.04.2019 2:13:49	3180590	3180590	N	Y
20	EX0	BESCPI	BESCPI	BEHAVIORAL SCORING PI	14.05.2011 19:39:59	10.11.2003 15:02:49	1680	1671998	N	N
43	EX0	BJ	BJ	BROKERJET	28.01.2013 20:03:29	14.05.2011 19:39:59	1671998	2219779	N	Y
51	EX0	CDCP	CDCP	CENTRALNI DEPOZITAR CENNYCH PAPIRU	26.04.2014 1:14:38	26.04.2014 1:14:38	2347228	2347228	N	Y
58	EX0	CRU	CRU	CENTRALNI REGISTR UVERU	16.04.2016 2:44:23	16.04.2016 2:44:23	2607174	2607174	N	N
62	EX0	CERTIS	CERTIS	CERTIS	01.11.2017 20:06:30	16.07.2016 3:33:49	2649283	2900083	N	N
18	EX0	CIC	CIC	CIC	14.05.2011 19:39:59	10.11.2003 15:02:49	1680	1671998	N	N
21	EX0	CIS	CIS	CIS	14.05.2011 19:39:59	10.11.2003 15:02:49	1680	1671998	N	N
4	EX0	CRM	CRM	CRM	14.05.2011 19:39:59	04.04.2003 17:54:59	2	1671998	N	N
8	EX0	DWH	DWH	CS DATAWAREHOUSE	14.05.2011 19:39:59	04.04.2003 17:54:59	2	1671998	N	N
7	EX0	ODS	ODS	CS OPERATIONAL DATASTORE	14.05.2011 19:39:59	04.04.2003 17:54:59	2	1671998	N	N
64	EX0	CGP	CGP	CUSTOMER GLOBAL POSITION	20.07.2019 1:57:09	15.07.2017 7:38:28	2843885	3230306	Y	N

SELECT

Filtrování – abychom mohli datovou sadu omezit, musíme využít podmínek v části **WHERE**.

Základní syntaxe:

```
SELECT
  <seznam sloupců>
FROM
  <schéma>.<tabulka>
WHERE
  <podmínka> (and / or další podmínka ...)
ORDER BY
  <seznam sloupců ASC / DESC>
;
```

Podmínka:

Syntaxe je <jméno sloupce> <operátor> <hodnota / jméno sloupce>. Podmínky se mohou závorkovat a využívat se mohou logické operátory AND a OR.

Operátory

Operátor:

Rovnosti a nerovnosti

Operátor	Význam
=	Rovnost
<> (!=)	Nerovnost
>	Větší než
>=	Větší či rovno
<	Menší než
<=	Menší či rovno

Další

Operátor	Význam
(not) like	Řetězec (ne)odpovídá masce
(not) in	Hodnota je (není) ve výčtu
(not) exists	Test (ne)existence v poddotazu
is (not) null	Test na (ne)prázdnost sloupce
between	Hodnota je mezi spodní a hodní mezí

Operátory

Úkol 1:

Vyberte záznam s ptori_key = 2.

Řešení:

*select * from dwh_owner.party_origins where ptori_key = 2;*

Úkol 2:

Vyberte záznamy s ptori_key větší než 10 a menší rovno 20.

Řešení:

*select * from dwh_owner.party_origins where ptori_key > 10 and ptori_key <= 20;*

Úkol 3:

Vyberte záznamy s ptori_key větší rovno 8 a menší rovno 15 nebo rovno 25.

Řešení:

*select * from dwh_owner.party_origins where (ptori_key >= 8 and ptori_key <= 15) or ptori_key = 25;*

Datové typy

Zatím jsme ve filtru používali jen čísla – nyní se seznámíme s dalšími datovými typy.

Datové typy:

Čísla:

1 1.025 -8 -.5

Řetězce (texty):

'textový řetězec'

Datum:

sysdate date '2026-03-25' '25.03.2026'

Datum

Abychom definovali datum, musíme – až na výjimky – napsat den, měsíc a rok. K tomu slouží funkce

to_date(<řetězec>, <formát>)

Formát pro den je typicky DD, pro měsíc MM a pro rok YYYY.

Pokud chceme jako datum zapsat 25.03.2026, tak musíme do <řetězec> zapsat tento řetězec v apostrofech a rovněž upravit formát tak, aby datu odpovídal. Formáty lze kombinovat dohromady a používat i znaky jako mezera, tečka, dvojtečka, pomlčka apod.

Tedy například níže uvedené zápasy jsou všechny validní.

to_date('20260325', 'YYYYMMDD')
to_date('2026-25-03', 'YYYY-DD-MM')
to_date('25.03.2026', 'DD.MM.YYYY')

Obdobně pokud chceme pracovat s časem, je nutné použít masku HH24 pro hodiny, MI pro minuty a SS pro vteřiny.

Tedy 25.03.2026 14:21:01 napíšeme např. jako *to_date('25.03.2026 14:21:01', 'DD.MM.YYYY HH24:MI:SS')*.

Datum

Úkol:

Vypište datum 19.5.1983 07:45:21, pro vypsání použijte tabulku PARTY_ORIGINS. Tedy místo * napište do části select pouze to_date funkci.

Řešení:

select to_date('19830519 07:45:21', 'YYYYMMDD HH24:MI:SS') from dwh_owner.party_origins;

Poznámka:
Pro výpis je možné použít speciální jednořádkovou tabulku DUAL.

select to_date(...) from dual;

Zkuste výsledek porovnat s

select to_date('19830519 07:45:21', 'YYYYMMDD HH24:MI:SS') from dual;

Datum

Převody mezi datovými typy:

Datum:

to_date(<řetězec>, <formát>)

Formát	Význam	Příklad	Výsledek
YYYY	Rok ve formátu „na čtyři znaky“	to_date('2026', 'YYYY')	1.3.2026
YY	Rok ve formátu „na dva znaky“	to_date('26', 'YY')	1.3.2026
RRRR	Rok ve formátu „na čtyři znaky“ 00-49 – 21. století, 50-99 – 20. století	to_date('26', 'RRRR') to_date('80', 'RRRR')	1.3.2026 1.3.1980
MON	Anglická třípísmenná zkratka pro měsíc	to_date('MAR', 'MON')	1.3.2026
MONTH	Anglický název pro měsíc	to_date('MARCH', 'MONTH')	1.3.2026
MM	Číselná zkratka pro měsíc 01-12	to_date('03', 'MM')	1.3.2026
DY	Anglická třípísmenná zkratka pro den	to_date('MON', 'DY')	day of week conflicts with Julian date
DY	Anglická třípísmenná zkratka pro den	to_date('SUN', 'DY')	1.3.2026
DAY	Anglický název pro den	to_date('SUNDAY', 'DAY')	1.3.2026
DD	Číselná zkratka pro den 01-31	to_date('02', 'DD')	2.3.2026
HH24	Číselná zkratka pro hodinu 00-23	to_date('13', 'HH24')	1.3.2026 13:00:00
HH (HH12)	Číselná zkratka pro hodinu 01-12	to_date('1', 'HH')	1.3.2026 1:00:00
MI	Číselná zkratka pro minuty 00-59	to_date('1', 'MI')	1.3.2026 0:01:00
SS	Číselná zkratka pro vteřiny 00-59	to_date('1', 'SS')	1.3.2026 0:00:01

Datum

Práce s datem:

datum + <číslo> - připočítání <číslo> dní k datu
datum - <číslo> - odečtení <číslo> dní od data

datum – datum – rozdíl ve dnech – 24 hodin = 1; 6 hodin = 0.25

add_months(datum, <číslo>) – přidá <číslo> měsíců k datu; může být i záporné

months_between(datum, datum) – vrací počet měsíců mezi dvěma daty

last_day(datum) – vrací poslední den v měsíci

trunc(datum, <maska>) – ořeže datum dle masky; maska odpovídá formátu z to_date, volí se však jen jedna

round(datum, <maska>) – zaokrouhlí datum dle masky

sysdate – vrací aktuální datum a čas

Datum

Příklady: (01_sql_datum.txt)

```
select to_date('25.03.2026', 'DD.MM.YYYY') + 55 from dual;   --19.5.2026

select to_date('25.03.2026', 'DD.MM.YYYY') - 55 from dual;   --29.1.2026

select to_date('25.03.2026 08:00', 'DD.MM.YYYY HH24:MI') - to_date('21.03.2026 14:00', 'DD.MM.YYYY HH24:MI') from dual;   -- 3.75

select add_months(to_date('25.03.2026', 'DD.MM.YYYY'), -3) from dual;   --25.12.2025

select months_between(to_date('25.03.2026', 'DD.MM.YYYY'), to_date('25.05.2026', 'DD.MM.YYYY')) from dual;    /* -2 */

select last_day(to_date('25.03.2026', 'DD.MM.YYYY')) from dual;    --31.3.2026

select trunc(to_date('25.03.2026', 'DD.MM.YYYY'), 'MM') from dual;    --1.3.2026

select round(to_date('25.03.2026', 'DD.MM.YYYY'), 'MM') from dual;   --1.4.2026

select sysdate from dual;    --25.03.2026 11:31:25

select round(sysdate, 'HH24') from dual;   --25.03.2026 12:00:00
```

Datum

Úkol:

Vypište zítřejší datum bez času?

Řešení:

```
select trunc(sysdate)+1 from dual;
```

Úkol:

Kolik minut zbývá od 25.03.2026 12:00 do 24.12.2026 18:00?

Řešení:

```
select (to_date('20261224 18:00', 'YYYYMMDD HH24:MI') - to_date('20250325 12:00', 'YYYYMMDD HH24:MI')) * 24 * 60 from dual;   -- 920520
```

Úkol:

Kdybych dnes podal výpověď, kdy mi skončí výpovědní lhůta? Lhůtu určete ještě dle starého Zákoníku práce.

Řešení:

```
select add_months(last_day(trunc(sysdate)), 2) from dual;
```

Řetězec

Převody mezi datovými typy:

Řetězec:

to_char(<datum>, <formát>)

Formát	Význam	Příklad	Výsledek
YYYY	Rok ve formátu „na čtyři znaky“	to_char(sysdate, 'YYYY')	2026
YY	Rok ve formátu „na dva znaky“	to_char(sysdate, 'YY')	26
MON	Anglická třípísmenná zkratka pro měsíc	to_char(sysdate, 'MON')	MAR
MONTH	Anglický název pro měsíc	to_char(sysdate, 'MONTH')	MARCH
MM	Číselná zkratka pro měsíc 01-12	to_char(sysdate, 'MM')	03
D	Pořadí dne v týdnu 1 – 7 (1 má neděle)	to_char(sysdate, 'D')	4
DY	Anglická třípísmenná zkratka pro den	to_char(sysdate, 'DY')	WED
DAY	Anglický název pro den	to_char(sysdate, 'DAY')	WEDNESDAY
DD	Číselná zkratka pro den 01-31	to_char(sysdate, 'DD')	25
HH24	Číselná zkratka pro hodinu 00-23	to_char(sysdate, 'HH24')	13
HH (HH12)	Číselná zkratka pro hodinu 01-12	to_char(sysdate, 'HH')	01
MI	Číselná zkratka pro minuty 00-59	to_char(sysdate, 'MI')	32
SS	Číselná zkratka pro vteřiny 00-59	to_char(sysdate, 'SS')	27

Řetězec

Převody mezi datovými typy:

Řetězec:

to_char(<datum>, <formát>, <jazyk>)

Výchozí jazyk a tím pádem i návratové hodnoty jsou v angličtině. Můžeme však vybrat i češtinu pomocí:

'NLS_DATE_LANGUAGE = CZECH'

*select to_char(sysdate, 'MONTH', **'NLS_DATE_LANGUAGE = CZECH'**) from dual;*

vrací hodnotu **BŘEZEN**.

Pro správné pořadí dne v týdnu, tedy 1 pro pondělí, je třeba spustit

alter session set nls_territory = 'CZECH REPUBLIC';

Řetězec

Úkol:

Jaký den bude za dva dny a jaký bude za měsíc?

Řešení:

```
select to_char(sysdate+2, 'DAY', 'NLS_DATE_LANGUAGE = CZECH') from dual;
select to_char(add_months(sysdate, 1), 'DAY', 'NLS_DATE_LANGUAGE = CZECH') from dual;
```

Hodnoty je možné vypsat i naráz:

```
select to_char(sysdate+2, 'DAY', 'NLS_DATE_LANGUAGE = CZECH'), to_char(add_months(sysdate, 1), 'DAY',
'NLS_DATE_LANGUAGE = CZECH') from dual;
```

Řetězec

Převody mezi datovými typy:

Řetězec:

to_char(<číslo>, <formát>)

Formát	Význam	Příklad	Výsledek
0	Vodící nula	to_char(1, '000')	001
9	Číslice či mezera	to_char(1, '999')	1
, nebo G	Oddělovač na specifické pozici	to_char(123456, '00,00,00')	12,34,56
. nebo D	Oddělovač desetinných míst	to_char(12.3456, '99.99')	12.35
FM	Odstranění mezer	to_char(12, 'FM9999')	12
S	Znaménko +/-	to_char(12, 'S9999')	+12

Řetězec

Převody mezi datovými typy:

Řetězec:

to_char(<číslo>, <formát>, <nls parametr>)

Výchozí formátování je tečka jako desetinný oddělovač a čárka jako oddělovač „tisíců“. Toto chování můžeme otočit pomocí

'NLS_NUMERIC_CHARACTERS = ",."'

Resp. obecně můžeme nastavit libovolné oddělovače – první je vždy desetinný, druhý „tisícový“.

select to_char(-1100.56,'9G990D00', 'NLS_NUMERIC_CHARACTERS = "#!"') from dual;

vrací hodnotu -1!100#56.

Abyste v řetězci mohli použít apostrof ('), musíte napsat dva (''). Tedy jméno Radar O'Reilly napíšeme v řetězci 'O''Reilly'. Proto jsou apostrofy za rovnítkem u NLS_NUMERIC_CHARACTERS zdvojené, jsou uvnitř řetězce.

Řetězec

Cvičení:

Co vypíše níže uvedený dotaz?

select """" from dual;

Řešení:

"

Řetězec

Úkol:

Naformátujte číslo 100000 tak, aby na výpisu bylo jako +100.000,00

Řešení:

```
select to_char(100000, 'S999G999D00','NLS_NUMERIC_CHARACTERS = '',.''') from dual;
```

Řetězec

Práce s řetězci:

řetězec || řetězec – spojení (konkatenace) dvou řetězců

lower(řetězec) – převede řetězec na malá písmena

upper(řetězec) – převede řetězec na velká písmena

initcap(řetězec) – převede první písmeno každého slova na velké a zbytek na malá

substr(řetězec, <start>, <počet>) – vrací <počet> znaků řetězce počínaje <start>. Je-li <start> záporný, pak se počítá zprava

instr(řetězec, podřetězec) – vrací pořadí prvního znaku podřetězce v řetězci

replace(řetězec, <co>, <za>) – zamění všechny výskyty <co> v řetězci za <za>

length(řetězec) – vrací délku řetězce

trim(řetězec) – ořeže všechny mezery na začátku a konci řetězce

lpad(řetězec, <délka>, <znak>) – doplní řetězec zleva znaky <znak> na celkovou délku <délka>; obdobně rpad() doplní zprava

Řetězec

Příklady: (02_sql_retezce.txt)

select 'Jiří' || 'Mareš' from dual; --**JiříMareš**

select lower('JiŘí') from dual; --**jiří**

select upper('JiŘí') from dual; --**JIŘÍ**

select initcap('JiŘí') from dual; --**Jiří**

select substr('Jiří Mareš', 3, 5) from dual; --**ří Ma**
select substr('Jiří Mareš', -5, 3) from dual; --**Mar**

select instr('Jiří Mareš', 'Mar') from dual; --**6**

select replace('Jirka Mareš', 'a' , 'u') from dual; --**Jirku Mureš**

select length('Jiří Mareš') from dual; **--10**

select trim(' Jiří Mareš ') from dual; **--Jiří Mareš**

select lpad('Jiří',8, '') from dual;* **/* ****Jiří */**

Řetězec

Úkol:

Vypište text „Tento měsíc se jmenuje '<měsíc>'.“?

Poznámka:

Například select 'Tento měsíc se jmenuje ''březen''. ' from dual;

Řešení:

```
select 'Tento měsíc se jmenuje ''' || trim(lower(to_char(sysdate, 'MONTH', 'NLS_DATE_LANGUAGE = CZECH'))) || '''.'
from dual;
select 'Tento měsíc se jmenuje ''' || trim(to_char(sysdate, 'month', 'NLS_DATE_LANGUAGE = CZECH')) || '''.' from dual;
```

Poznámka:

Můžete si všimnout dvou věcí. Za prvé masky, které vrací řetězce, vrací pevný počet znaků, tj. na konci vrací mezery. A za druhé mají následující chování: Je-li první písmeno masky malé, vrací text jako lower. Jinak se řídí druhým písmenem; je-li velké, vrací upper, pro malé initcap. Tedy 'MONTH', 'month' i 'Month' vrací jiné výsledky!

Řetězec

Úkol:

Vypište abecedně jména (pt_party_name) všech klientů z LDAPu tak, aby byl sloupec doplněný zleva hvězdičkami na celkovou délku 100 znaků.

Řešení:

```
select
 lpad(pt_party_name, 100, '*')
from
 dwh_owner.parties
where
 ptori_key = 32;
```

Řetězec

Úkol:

Vypište abecedně jména (pt_party_name) všech klientů z LDAPu tak, aby byl sloupec doplněný rovnoměrně zleva i zprava hvězdičkami na celkovou délku 100 znaků. Pro zaokrouhlení použijte funkci round(), a operátory + (sčítání), - (odčítání), * (násobení) a / (dělení).

Řešení:

```
select
  rpad(lpad(pt_party_name, length(pt_party_name) + round((100-length(pt_party_name))/2), '*'), 100, '*')
from
  dwh_owner.parties
where
  ptori_key = 32;
```

Řetězec

Porovnání řetězců:

U typu podmínek ve WHERE klauzuli jsme zmiňovali funkci LIKE. Na tu se podíváme blíže nyní.

Syntaxe:

<sloupec / řetězec> **LIKE** <maska>

Maska je řetězec, který může obsahovat dva speciální znaky.

% - zástupný znak za libovolný počet jakýchkoliv znaků
_ - zástupný znak za jeden jakýkoliv znak

Tedy například

'starbank' odpovídá masce '%bank', 'star%', '_ta%an_' či '_____', ale neodpovídá 'Starbank'

Poznámka:

Pokud byste potřebovali porovnávat řetězec na znak % či _, je nutné využít tzv. escape character.

Např. LIKE '%account!_status%' ESCAPE '!'

Řetězec

Úkol:

Vypište všechny záznamy z tabulky dwh_owner.account_types, jejichž popisek obsahuje slovo starbank (nebudeme rozlišovat velká a malá písmena).

Řešení:

```
select * from dwh_owner.account_types where lower(acctp_desc) like '%starbank%';
```

Úkol:

Vypište všechny záznamy z tabulky dwh_owner.account_types, jejichž popisek neobsahuje ani slovo účet, ani slovo účty (nebudeme rozlišovat velká a malá písmena).

Řešení:

```
select * from dwh_owner.account_types where lower(acctp_desc) not like '%účet%'
and lower(acctp_desc) not like '%účty%';
```

Úkol:

Vypište všechny záznamy z tabulky dwh_owner.products, jejichž jméno obsahuje %.

Řešení:

```
select * from dwh_owner.products where prod_name like '%!%%' escape '!';
```

Číslo

Převody mezi datovými typy:

Číslo:

to_number(<řetězec>, <formát>)

Formát	Význam	Příklad	Výsledek
0	Vodící nula	to_number(' 001', '000')	1
9	Číslice či mezera	to_number(' 1', '999')	1
, nebo G	Oddělovač na specifické pozici	to_number('12,34,56', '00,00,00')	123456
. nebo D	Oddělovač desetinných míst	to_number('12.3456', '99.9999')	12.3456
FM	Odstranění mezer	to_number(' 12', 'FM9999')	12
S	Znaménko +/-	to_number('+1200', 'S9999')	1200

Číslo

Úkol:

Vypište řetězec „17.6“ jako číslo pomocí funkce to_number;

Řešení:

select to_number('17.6') from dual;

select to_number('17.6', '99.9') from dual;

Úkol:

Vypište řetězec „+17,6“ jako číslo pomocí funkce to_number;

Řešení:

select to_number('+17,6', 'S99D9', 'NLS_NUMERIC_CHARACTERS='',.''') from dual;

Číslo

Práce s čísly:

číslo * / + - číslo – vynásobí (vydělí, přičte, resp. odečte) dvě čísla

trunc(číslo, <počet míst>) – ořeže číslo na <počet míst> desetinných míst

round(číslo, <počet míst>) – zaokrouhlí číslo na <počet míst> desetinných míst

ceil(číslo) – vrátí nejbližší vyšší či rovné celé číslo (tzv. horní celou část)

floor(číslo) – vrátí nejbližší nižší či rovné celé číslo (tzv. dolní celou část)

abs(číslo) – vrátí absolutní hodnotu čísla; tj. kladnou hodnotu

mod(číslo, <modulo>) – vrátí zbytek po dělení <modulo>

power(číslo, <mocnina>) – vrátí <mocnina>-ntou mocninu čísla

sqrt(číslo) – druhou odmocninu čísla

Dále funkce exponenciální (exp), logaritmické (ln, log), goniometrické (sin, cos, tan), cyklometrické (asin, acos, atan), hyperbolické (sinh, cosh, tanh) apod.

Číslo

Příklady: (03_sql_cislo.txt)

```
select trunc(2.55, 1) from dual;   --2.5

select round(2.55, 1) from dual;   --2.6

select ceil(2.55) from dual;   --3
select ceil(-2.55) from dual;   /* -2 */

select floor(2.55) from dual;   --2
select floor(-2.55) from dual;   /* -3 */

select abs(-2) from dual;   --2

select mod(15, 4) from dual;   --3

select power(2, 4) from dual;   --16

select sqrt(16) from dual;   --4
```

Motivační příklad

Kolik nových klientů za Starbanku bylo založeno nebo zrušeno v lednu 2026 a jaký je přírůstek / úbytek v procentech oproti stavu k ultimo prosince 2025.

Co už umíme?

- klienti Starbank mají proti_key = 2

Co nevíme?

- klient byl založen nebo zrušen v daném měsíci

Pro jednoduchost předpokládejme, že nový klient je ten unifikovaný klient, který existuje v daném měsíci a v minulém neexistoval. Obdobně, smazaný klient je smazaný v daném měsíci a v minulém měsíci smazaný nebyl. Klienty, kteří byli v prosinci založeni i zrušeni, budeme ignorovat.

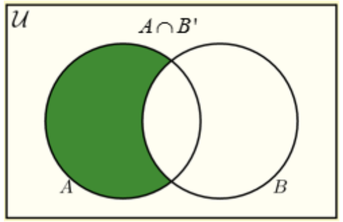
Množinové operace

Abychom poznali, který klient v daném měsíci existuje a v minulém neexistuje, potřebujeme mít dvě množiny dat. V první budeme mít všechny klienty platné v průběhu ledna 2026, v druhé množině pak platné klienty v prosinci 2025.

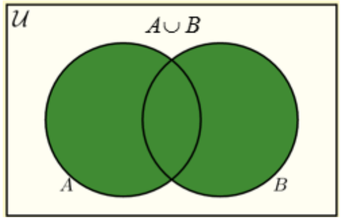
Obě tyto množiny jsou reprezentovány vlastním dotazem (=selectem).

S těmito množinami můžeme nadále pracovat – dělat jejich průnik, rozdíl či spojení.

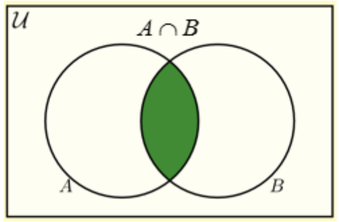
MINUS (rozdíl množin)



UNION (spojení)



INTERSECT (průnik)



Množinové operace

Aby bylo možné provádět jednotlivé operace, musí mít datové sety stejnou strukturu, tj. stejný počet sloupců a stejné (či alespoň převoditelné) datové typy.

Příklad:

```
select
 accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 201
intersect
select
 accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 204;
```

ACCST_KEY
11
12
13
14
15
2381
2382

Množinové operace

Příklad:

```
select
 accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 201
minus
select
 accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 204;
```

ACCST_KEY
16
17
18
19
1744
1745
1746

Množinové operace

Příklad:

```
select
 accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 201
union
select
 accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 204;
```

ACCST_KEY
10
11
12
13
14
15
16
17
18
19
1744
1745
1746
2381
2382

Množinové operace

Můžete si všimnout, že všechny tři operace s množinami mají následující vlastnosti:

- Data jsou seřazená
- Každá hodnota se vyskytuje ve výsledku jednou

Pokud se podíváme na první množinu dat, tyto „duplicity“ obsahuje, viz dotaz níže.

```
select
 accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 201;
```

ACCST_KEY
14
14
14
14
14
14
14
14
14
14
14
14

Množinové operace

Pokud nás zajímají jen unikátní hodnoty, je možné v dotazu použít klíčové slovo DISTINCT. Můžete si všimnout, že zde výsledek setříděný není.

```
select
 DISTINCT
  accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 201;
```

ACCST_KEY
14
15
11
1744
12
17
18
2382
10
13
2381
1746
19
1745
16

Množinové operace

Někdy je naopak vhodné zachovat duplicity i v případě spojování. V tomto případě se klíčové slovo UNION rozšíří o ALL.

Příklad:

```
select
 accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 201
UNION ALL
select
 accst_key
from
 dwh_owner.all_accounts
where
 acctp_key = 204;
```

ACCST_KEY
14
14
14
14
14
14
14
14
14
14
14
14
14
14
14
14

Množinové operace

Úkol:

Vypište všechny identifikátory nesmazaných poboček (org_unified_id v tabulce dwh_owner.organizations), které existují jak se zdrojem (org_source_system_id) 'SB0', tak se zdrojem 'EX0'.

Řešení:

```
select
 org_unified_id
from
 dwh_owner.organizations
where
 org_deleted_flag = 'N' and
 org_source_system_id = 'SB0'
intersect
select
 org_unified_id
from
 dwh_owner.organizations
where
 org_deleted_flag = 'N' and
 org_source_system_id = 'EX0';
```

Množinové operace

Úkol:

Vypište všechny identifikátory nesmazaných poboček (org_unified_id v tabulce dwh_owner.organizations), které existují buď se zdrojem (org_source_system_id) 'SB0', nebo se zdrojem 'EX0'.

Řešení:

```
select
  org_unified_id
from
  dwh_owner.organizations
where
  org_deleted_flag = 'N' and
  org_source_system_id = 'SB0'
union
select
  org_unified_id
from
  dwh_owner.organizations
where
  org_deleted_flag = 'N' and
  org_source_system_id = 'EX0';
```

Množinové operace

Úkol:

Vypište všechny identifikátory nesmazaných poboček (org_unified_id v tabulce dwh_owner.organizations), které existují se zdrojem (org_source_system_id) 'SB0', ale neexistují se zdrojem 'EX0'.

Řešení:

```
select
 org_unified_id
from
 dwh_owner.organizations
where
 org_deleted_flag = 'N' and
 org_source_system_id = 'SB0'
minus
select
 org_unified_id
from
 dwh_owner.organizations
where
 org_deleted_flag = 'N' and
 org_source_system_id = 'EX0';
```

Motivační příklad

Úkol:

Vypište všechny unifikované klienty, kteří přísluší starbankové instanci, a byli aktivní (tj. nesmazaní) v lednu 2026.

Poznámka:

Historizace klientů je v tabulce PARTY_HISTORY a jejich časová platnost je vždy od PTH_VALID_FROM a PTH_VALID_TO. Nesmazané klienty poznáme dle PTH_DELETED_FLAG = 'N'. Unifikovaný klient je uložen v atributu PT_UNIFIED_KEY.

Podmínku <datum> >= PTH_VALID_FROM and <datum> <= PTH_VALID_TO je možné nahradit operátorem between, tj.

<datum> between PTH_VALID_FROM and PTH_VALID_TO

Řešení:

```
select
  distinct
    pt_unified_key
from
  dwh_owner.party_history
where
  to_date('20260131', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
  ptori_key = 2 and
  pth_deleted_flag = 'N';
```

Motivační příklad

Úkol:

Nyní porovnejte stav (tj. udělejte rozdíl) s prosincem 2025.

Řešení:

```
select
 pt_unified_key
from
 dwh_owner.party_history
where
 to_date('20260131', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
 ptori_key = 2 and
 pth_deleted_flag = 'N'
minus
select
 pt_unified_key
from
 dwh_owner.party_history
where
 to_date('20251231', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
 ptori_key = 2 and
 pth_deleted_flag = 'N';
```

Motivační příklad

Kolik nových klientů za Starbanku bylo založeno nebo zrušeno v lednu 2026 a jaký je přírůstek / úbytek v procentech oproti stavu k ultimo prosince 2025.

Co už umíme?

- klienti Starbank mají proti_key = 2
- klient byl založen nebo zrušen v daném měsíci

Co ještě neumíme?

- zjistit počty záznamů

Agregační funkce

Agregační funkce jsou v funkce, které umožňují seskupit vybrané řádky dotazu a spočítat nad nimi výsledek určité aritmetické
nebo statistické funkce. K agregování záznamů slouží klauzule **GROUP BY**.

Základní syntaxe:

```
SELECT
  <agregované sloupce>,
  <agregující sloupce>
FROM
  <schéma>.<tabulka>
GROUP BY
   <agregující sloupce>
ORDER BY
  <seznam sloupců ASC / DESC>
;
```

Agregační funkce

Agregační funkce:

Funkce	Význam
AVG	Průměrná hodnota
COUNT	Počet hodnot
MAX	Maximální hodnota
MEDIAN	Medián
MIN	Minimální hodnota
SUM	Celková suma

Poznámka:

Pokud provádíme agregaci na úrovni celé tabulky a nikoliv jednotlivých sloupců, výraz GROUP BY se nepoužije. Ve výsledku však musí být výhradně agregační funkce.

Při agregování se vždy ignorují hodnoty **null**.

Agregační funkce

Příklad:

Každý klient v SAPu má svůj osmiciferný identifikátor. Posledních pět číslic odpovídá CENu / EXTu. Ověřte, jestli je tato pětice unikátní . Tedy zjistěte četnosti jednotlivých pětic a ty vypište sestupně.

```
select
  count(1) as pocet,
  substr(pt_source_id, 4) as cen
from
  dwh_owner.parties
where
  ptori_key = 11
group by
  substr(pt_source_id, 4)
order by
  1 desc, 2;
```

Agregační funkce

Úkol:

Vypište počet všech nesmazaných poboček.

Řešení:

```
select count(1) as pocet from dwh_owner.organizations where org_deleted_flag = 'N';
```

Úkol:

Vypište počet všech nesmazaných poboček podle zdroje (org_source_system_id).

Řešení:

```
select
  count(1) as pocet,
  org_source_system_id
from
  dwh_owner.organizations
where
  org_deleted_flag = 'N'
group by
  org_source_system_id
order by
  1;
```

Agregační funkce

Někdy je vhodné, například při hledání duplicit, mít na agregované záznamy dodatečné požadavky. Například vybrat pouze ty záznamy, které se vyskytují alespoň dvakrát. K tomu slouží klauzule **HAVING**.

Základní syntaxe:

```
SELECT
  <agregované sloupce>,
  <agregující sloupce>
FROM
  <schéma>.<tabulka>
GROUP BY
   <agregující sloupce>
HAVING
  <agregační  funkce> <podmínka>
ORDER BY
  <seznam sloupců ASC / DESC>
;
```

Agregační funkce

Příklad:

Každý klient v SAPu má svůj osmiciferný identifikátor. Posledních pět číslic odpovídá CENu / EXTu. Vypište ty záznamy, které nejsou unikátní .

```
select
  count(1) as pocet,
  substr(pt_source_id, 4) as cen
from
  dwh_owner.parties
where
  ptori_key = 11
group by
  substr(pt_source_id, 4)
having
  count(1) > 1
order by
  1 desc, 2;
```

Agregační funkce

Úkol:

Vypište ty nesmazané (org_unified_id) pobočky, které evidujeme v tabulce alespoň třikrát.

Řešení:

```
select
  count(1) as pocet,
  org_unified_id
from
  dwh_owner.organizations
where
  org_deleted_flag = 'N'
group by
  org_unified_id
having
  count(1) > 2
order by
  1 desc, 2;
```

Agregační funkce

Úkol:

Vypište ty nesmazané (org_unified_id) pobočky, které evidujeme alespoň ze tří zdrojů (org_source_system_id).

Řešení:

```
select
  count(1) as pocet,
  org_unified_id
from
  (
    select
      distinct
        org_unified_id,
        org_source_system_id
    from
      dwh_owner.organizations
    where
      org_deleted_flag = 'N'
  )
group by
  org_unified_id
having
  count(1) > 2
order by
  1 desc, 2;
```

Ukázkový příklad

Ukázkový příklad:

Vypište, kolik je nesmazaných klientů Starbanku dle pohlaví a věku (rozmezí 0-17, 18-64 a 65 a více).

Nejprve se podíváme na tabulku PERSON_GENDERS.

PSGEN_KEY	PSGEN_SOURCE_SYSTEM_ID	PSGEN_SOURCE_ID	PSGEN_DESC
1	EX0	F	ŽENA
2	EX0	M	MUŽ
3	EX0	O	ORGANIZACE
-2	XNA	XER	INVALID
-1	XNA	XNA	N/A

Klíče (hodnoty) psgen_key, které jsou v PARTIES, by nám mnoho neřekly, proto se podíváme, jakým způsobem je můžeme v dotazu upravit. Použijeme funkci **DECODE**.

Decode

Syntaxe:

DECODE(zdroj, hodnota_1, výsledek_1, hodnota_2, výsledek_2, …, hodnota_n, výsledek_n, výsledek_else)

Vyhodnocování probíhá tak, že se nejprve zjistí, jestli je zdroj roven hodnota_1. Pokud ano, použije se výsledek_1. Pokud ne, dojde k dalšímu porovnání (tedy na hodnota_2) a opět se buď použije výsledek_2 v případě rovnosti nebo se porovnává dál. Pokud nedojde k rovnosti ani u hodnota_n, použije se výsledek_else.

V našem případě budeme pracovat jen s hodnotami psgen_key 1 a 2, a proto použijeme jednoduchou konstrukci

DECODE(psgen_key, 1, 'Žena', 'Muž').

Decode

Úkol:

Napište pomocí funkce decode, jestli je zadané datum víkend či není.

Nezapomeňte nejprve spustit příkaz

alter session set nls_territory = "CZECH REPUBLIC";

Řešení:

```
select decode(to_char(sysdate, 'D'), '6', 'Víkend', '7', 'Víkend', 'Všední den')  from dual;
```

Case

Zbývá nám určení věkových hranic. Stáří v letech zjistíme pomocí pt_birth_date v porovnání se sysdate.

Například

trunc(months_between(trunc(sysdate), pt_birth_date) / 12)

Podíváme se na další funkci a tou je **CASE**.

Case

Syntaxe:

```
CASE
  WHEN
    podmínka_1
  THEN
    hodnota_1
  WHEN
    podmínka_2
  THEN
    hodnota_2
...
  WHEN
    podmínka_n
  THEN
    hodnota_n
  ELSE
    hodnota_else
END
```

Vyhodnocení je obdobné jako u DECODE, CASE je jen robustnější konstrukce.

Case

V našem případě použije case takto:

```
CASE
  WHEN
    trunc(months_between(trunc(sysdate), pt_birth_date) / 12) < 18
  THEN
    'méně než 18'
  WHEN
    trunc(months_between(trunc(sysdate), pt_birth_date) / 12) >= 65
  THEN
    '65 a více'
  ELSE
    'mezi 18 a 64'
END
```

Case

Úkol:

Napište pomocí funkce case, jestli je zadané datum víkend či není.

Řešení:

```
select
 case
  when
   to_char(sysdate, 'D') = '6' or to_char(sysdate, 'D') = '7'
  then
   'Víkend'
  else
   'Všední den'
 end
from
 dual;
```

Ukázkový příklad *(04_sql_klienti.txt)*

Vypište, kolik je nesmazaných klientů Starbanku dle pohlaví a věku (rozmezí 0-17, 18-64 a 65 a více).

```
select
 count(1),
 decode(psgen_key, 1, 'Žena', 'Muž'),
 case
  when
   trunc(months_between(trunc(sysdate), pt_birth_date) / 12) < 18
  then
   'méně než 18'
  when
   trunc(months_between(trunc(sysdate), pt_birth_date) / 12) >= 65
  then
   '65 a více'
  else
   'mezi 18 a 64'
 end
from
 dwh_owner.parties
where
 ptori_key = 2 and
 pt_deleted_flag = 'N' and
 psgen_key in (1, 2)
group by
 decode(psgen_key, 1, 'Žena', 'Muž'),
 case
  when
   trunc(months_between(trunc(sysdate), pt_birth_date) / 12) < 18
  then
   'méně než 18'
  when
   trunc(months_between(trunc(sysdate), pt_birth_date) / 12) >= 65
  then
   '65 a více'
  else
   'mezi 18 a 64'
 end
order by
 2, 3 desc;
```

Poddotazy

Někdy je vhodné pracovat s poddotazy, tedy se selectem ne přímo z tabulky, ale z jiného selectu.

Syntaxe:

```
select
 <vybrane_sloupce>
from
 (
  select
   <sloupce>
  from
   tabulka
 );
```

Na první pohled nemusí být vidět praktické využití, podívejme se na konkrétní příklad.

Předpokládejme, že máme vypsat první tři dny v roce.

Poddotazy

Standardně bychom použili dotaz

```
select
 trunc(sysdate, 'YYYY') as prvni,
 trunc(sysdate, 'YYYY')+1 as druhy,
 trunc(sysdate, 'YYYY')+2 as treti
from
 dual;
```

Transformaci trunc na sysdate musíme použít třikrát. Níže je příklad, kde je použití transformace jen jediné.

```
select
 prvni,
 prvni + 1 as druhy,
 prvni + 2 as treti
from
 (
  select
   trunc(sysdate, 'YYYY') as prvni
  from
   dual
 );
```

Všimněte si, že vybrané výrazy si mohu pojmenovat. Klíčové slovo **as** je nepovinné, stačí, když je za výrazem mezera a alias.

Poddotazy

Úkol:

Vypište posledních pět dní příštího měsíce.

Řešení:

```
select
 posledni,
 posledni - 1 as predposledni,
 posledni - 2 as predpredposledni,
 posledni - 3 as predpredpredposledni,
 posledni - 4 as predpredpredpredposledni
from
 (
  select
   add_months(last_day(trunc(sysdate)), 1) as posledni
  from
   dual
 );
```

Ukázkový příklad *(04_sql_klienti.txt)*

Úkol:

Upravte řešení tak, abyste využili poddotaz a nemuseli tak používat výrazy v části v „group by“.

Řešení:

```
select
 count(1) as pocet,
 pohlavi,
 hranice
from
 (
  select
   decode(psgen_key, 1, 'Žena', 'Muž') as pohlavi,
   case
    when
     trunc(months_between(trunc(sysdate), pt_birth_date) / 12) < 18
    then
     'méně než 18'
    when
     trunc(months_between(trunc(sysdate), pt_birth_date) / 12) >= 65
    then
     '65 a více'
    else
     'mezi 18 a 64'
   end as hranice
  from
   dwh_owner.parties
  where
   ptori_key = 2 and
   pt_deleted_flag = 'N' and
   psgen_key in (1, 2)
 )
group by
 pohlavi, hranice
order by
 2, 3 desc;
```

Ukázkový příklad *(04_sql_klienti.txt)*

Úkol:

Upravte řešení tak, abyste využili ještě další poddotaz a nemuseli tak používat výpočet věku víckrát.

Řešení:

```
select
 count(1) as pocet, pohlavi, hranice
from
 (
  select
   pohlavi,
   case
    when
     vek < 18
    then
     'méně než 18'
    when
     vek >= 65
    then
     '65 a více'
    else
     'mezi 18 a 64'
   end as hranice
  from
   (
    select
     decode(psgen_key, 1, 'Žena', 'Muž') as pohlavi,
     trunc(months_between(trunc(sysdate), pt_birth_date) / 12) as vek
    from
     dwh_owner.parties
    where
     ptori_key = 2 and pt_deleted_flag = 'N' and psgen_key in (1, 2)
   )
 )
group by
 pohlavi, hranice
order by 2, 3 desc;
```

Příklad

Úkol:

Vypište nejnižší, nejvyšší a průměrnou dobu v letech, po jakou jsou u nás unifikovaní klienti v závislosti na pohlaví.

Poznámka:

Hledaný atribut je pt_client_cs_from_date. Použijte filtr na pt_client_cs_from_date > pt_birth_date, psgen_key in (1,2), pt_deleted_flag = 'N', ptori_key = 4 a ptstat_cs_key = 6

Řešení:

```
select
 pohlavi,
 min(delka),
 avg(delka),
 max(delka)
from
 (
  select
   decode(psgen_key, 1, 'Žena', 'Muž') as pohlavi,
   trunc(months_between(trunc(sysdate), pt_client_cs_from_date) / 12) as delka
  from
   dwh_owner.parties
  where
   ptori_key = 4 and
   ptstat_cs_key = 6 and
   pt_deleted_flag = 'N' and
   psgen_key in (1, 2) and
   pt_client_cs_from_date > pt_birth_date
 )
group by
 pohlavi;
```

NULL

V případě agregačních funkcí bylo uvedeno, že jsou ignorovány hodnoty NULL. S těmi jsme se dosud nesetkali.

Co znamená NULL a jak s tou hodnotou pracovat?

NULL je prázdná, nevyplněná, neznámá hodnota. S NULL hodnotou se nedá běžně pracovat, není možné ji ani běžnými prostředky porovnávat. V případě podmínek je nutné použít

IS NULL či **IS NOT NULL**

Pokud pro nás NULL není vyhovující hodnota, a rádi bychom prázdné hodnoty ošetřili, můžeme použít dvě funkce.

NVL a **NVL2**

NVL

Syntaxe:

NVL(<zdroj>, <hodnota>)

Funkce NVL zjistí, jestli je <zdroj> roven NULL a pokud ano, použije <hodnota>. V opačném případě vrátí <zdroj>.

NVL2(<zdroj>, <notnull_hodnota>, <null_hodnota>)

Obdobně funguje i NVL2. Pokud je <zdroj> roven NULL, použije <null_hodnota>, jinak <notnull_hodnota>. Zde je velmi důležité pořadí argumentů!

Poznámka:

Při práci s NULL hodnotami se můžete setkat rovněž s funkcí COALESCE, která má libovolný počet parametrů, a vrací hodnotu prvního „nenullového“ z nich.

Příklad

Úkol:

Některé starší transakce nemají vyplněné políčko příjemce. Zkuste na ně aplikovat funkci nvl a nahraďte je za text „neznámý příjemce“.

Poznámka:

Budeme pracovat s tabulkou ACCOUNT_TRANSACTIONS. Použité filtry budou acctp_key = 101 a acctrn_accounting_date bude od 1.1.2026 do 3.1.2026. Příjemce je v poli acctrn_recipient_name.

Řešení:

```
select
  nvl(acctrn_recipient_name, 'neznámý příjemce') as prijemce
from
  dwh_owner.account_transactions
where
  acctp_key = 101 and
  acctrn_accounting_date between to_date('20260101', 'YYYYMMDD') and to_date('20260103', 'YYYYMMDD');
```

Příklad

Úkol:

Zkuste nyní na stejnou skupinu záznamů aplikovat funkci nvl2, kde v případě vyplněné hodnoty použijete „známý příjemce“. Kolik hodnot je v jednotlivých dnech vyplněných a kolik ne?

Řešení:

```
select
 datum,
 prijemce,
 count(1) as pocet
from
 (
  select
   acctrn_accounting_date as datum,
   nvl2(acctrn_recipient_name, 'známý příjemce', 'neznámý příjemce') as prijemce
  from
   dwh_owner.account_transactions
  where
   acctp_key = 101 and
   acctrn_accounting_date between to_date('20260101', 'YYYYMMDD') and to_date('20260103', 'YYYYMMDD')
 )
group by
 datum, prijemce
order by 1, 2;
```

Příklad

Poznámka:

V tomto případě bychom nemuseli využívat funkci nvl2, postačil by nám case.

```
select
  acctrn_accounting_date as datum,
  sum(case when acctrn_recipient_name is null then 1 else 0 end) as nevyplnene,
  sum(case when acctrn_recipient_name is not null then 1 else 0 end) as vyplnene
from
  dwh_owner.account_transactions
where
  acctp_key = 101 and
  acctrn_accounting_date between to_date('20260101', 'YYYYMMDD') and to_date('20260103', 'YYYYMMDD')
group by
  acctrn_accounting_date
order by 1;
```

Příklad

Poznámka:

A dokonce i prostý count!

```
select
  acctrn_accounting_date as datum,
  count(1) - count(acctrn_recipient_name) as nevyplnene,
  count(acctrn_recipient_name) as vyplnene
from
  dwh_owner.account_transactions
where
  acctp_key = 101 and
  acctrn_accounting_date between to_date('20260101', 'YYYYMMDD') and to_date('20260103', 'YYYYMMDD')
group by
  acctrn_accounting_date
order by 1;
```

Poznámka:

Jak se liší počty vrácených řádků v prvním a druhé (třetím) dotazu?

IN

V ukázkovém případu byla použita funkce IN. Nyní se na ni blíže podíváme.

Základní syntaxe:

<zdroj> IN (<hodnota_1>, <hodnota_2>, ..., <hodnota_n>)

IN je v tomto kontextu je zkratkou pro

<zdroj> = <hodnota_1> OR <zdroj> = <hodnota_2> OR ... OR <zdroj> = <hodnota_n>

Poznámka:

Počet hodnot je omezen na číslo 1000.

IN

IN však může pracovat i se selectem.

Základní syntaxe:

<zdroj> IN (select <hodnoty> from tabulka)

V tomto případě se vyberou všechny záznamy, u nichž se zdroj rovná kterémukoliv záznamu z <hodnoty>. Návratové záznamy z <hodnoty> mohou být duplicitní, výsledné řádky se <zdroj> ale namnoženy nebudou.

Poznámka:

Pokud používáte v IN klauzuli select, pak počet záznamů (vrácených tímto selectem) není nikterak omezen, tj. omezení na 1000 záznamů zde neplatí.

Příklad

Úkol:

Vypište počet unifikovaných a nesmazaných klientů, kteří se narodili ve stejný den jako klient s cluidem (pt_unique_party_id) 1999-08-24-23.16.53.033522.

Řešení:

```
select
 count(*)
from
 dwh_owner.parties
where
 ptori_key = 4 and
 pt_deleted_flag = 'N' and
 pt_birth_date in
  (
   select
    pt_birth_date
   from
    dwh_owner.parties
   where
    ptori_key = 4 and
    pt_unique_party_id = '1999-08-24-23.16.53.033522'
  );
```

Příklad

Poznámka:

Pokud máte naprostou jistotu, že dílčí select vrací **PRÁVĚ** jeden záznam, pak je možné namísto IN použít prosté =.

```
select
 count(*)
from
 dwh_owner.parties
where
 ptori_key = 4 and
 pt_deleted_flag = 'N' and
 pt_birth_date =
  (
   select
    pt_birth_date
   from
    dwh_owner.parties
   where
    ptori_key = 4 and
    pt_unique_party_id = '1999-08-24-23.16.53.033522'
  );
```

IN

IN je možné použít i na sadu atributů.

<u>Základní syntaxe:</u>

(<zdroj_1>, <zdroj_2>, ..., <zdroj_n>) IN ((<hodnota_1_1>, <hodnota_1_2>, ..., <hodnota_1_n>), (<hodnota_2_1>, <hodnota_2_2>, ..., <hodnota_2_n>, ..., (<hodnota_m_1>, <hodnota_m_2>, ..., <hodnota_m_n>)))

Obdobně i

(<zdroj_1>, <zdroj_2>, ..., <zdroj_n>) IN (select <hodnoty_1>, <hodnoty_2>, ..., <hodnoty_n> from tabulka)

Příklad

Ukázkový příklad:

```
select
 *
from
 dwh_owner.accounts
where
 (acctp_key, ptori_key) in ((101, 1), (101, 2), (202, 2));
```

ACC_KEY	ACC_MASTER_KEY	ORG_KEY	CLOSETP_KEY	ACCSCH_KEY	PT_KEY	PTORI_KEY	ACCTP_KEY	ACCTP_MASTER_KEY	ACCLS_KEY	ACCCLS_DISPO_RESTR_KEY	ACCLS_WORST_KEY	CURR_KEY
25723	-1	884	-1	-1	9025781	1	101	-1	-1000001	-2000001	-1000001	39
25725	-1	1180	-1	-1	10660330	1	101	-1	5	6	5	39
25726	-1	3933	-1	1	22683057	2	101	-1	3	4	3	39
25727	-1	1335	-1	-1	11641942	1	101	-1	7	8	44	39
25728	-1	3643	-1	1	471247628	2	101	-1	3	4	5	39
25729	-1	1209	-1	-1	10832321	1	101	-1	1	2	1	39
25730	-1	1278	-1	-1	11171064	1	101	-1	-1000001	-2000001	-1000001	39
25731	-1	432441	-1	1	25227281	2	101	-1	1	2	1	39
25732	-1	3684	-1	1	211936947	2	101	-1	3	4	3	39
25733	-1	3768	-1	1	212049332	2	101	-1	3	4	3	39
25734	-1	741	-1	-1	8489986	1	101	-1	1	47	3	39
25735	-1	878	-1	-1	9001052	1	101	-1	1	47	1	39
25736	-1	1230	-1	-1	10947686	1	101	-1	1	47	1	39
25738	-1	3632	-1	1	24081216	2	101	-1	1	2	1	39
25739	-1	1199	-1	-1	10782229	1	101	-1	1	2	1	39

Příklad

Úkol:

Zkuste upravit minulý příklad a vypište počet všech klientů, kteří se narodili ve stejný den (ale jakýkoliv rok) a mají stejné křestní jméno jako klient s cluidem 1999-08-24-23.16.53.033522.

Řešení:

```
select
  count(1)
from
  dwh_owner.parties
where
  ptori_key = 4 and
  pt_deleted_flag = 'N' and
  (to_char(pt_birth_date, 'MMDD'), upper(pt_first_name)) in
    (
      select
        to_char(pt_birth_date, 'MMDD'),
        upper(pt_first_name)
      from
        dwh_owner.parties
      where
        ptori_key = 4 and
        pt_unique_party_id = '1999-08-24-23.16.53.033522'
    );
```

Příklad

Poznámka:

I zde bychom mohli zcela analogicky použít namísto IN prosté =.

```
select
 count(1)
from
 dwh_owner.parties
where
 ptori_key = 4 and
 pt_deleted_flag = 'N' and
 (to_char(pt_birth_date, 'MMDD'), upper(pt_first_name)) =
  (
   select
    to_char(pt_birth_date, 'MMDD'),
    upper(pt_first_name)
   from
    dwh_owner.parties
   where
    ptori_key = 4 and
    pt_unique_party_id = '1999-08-24-23.16.53.033522'
  );
```

IN

A ještě jedno speciální použití. Předpokládejme, že nás zajímají všechny vztahy klienta (party_relations) s pt_key = 3133711, ale nevíme, jestli je klient v tabulce na straně objektu či subjektu.

Pak nemusíme psát

```
select
 *
from
 dwh_owner.party_relations
where
 pt_object_key = 3133711 or
 pt_subject_key = 3133711;
```

Ale lze napsat

```
select
 *
from
 dwh_owner.party_relations
where
 3133711 in (pt_object_key, pt_subject_key);
```

Motivační příklad

Kolik nových klientů za Starbanku bylo založeno nebo zrušeno v lednu 2026 a jaký je přírůstek / úbytek v procentech oproti stavu k ultimo prosince 2025.

Co už umíme?

- klienti Starbank mají proti_key = 2
- klient byl založen nebo zrušen v daném měsíci
- zjistit počty záznamů

Co ještě neumíme?

- spojit výsledky dohromady

JOIN

Nyní se podíváme, jak dokážeme pracovat s více tabulkami najednou. Například potřebujeme k číslu účtu starbankového běžného účtu vypsat i jeho majitele. Víme, že v tabulce ACCOUNTS je odkaz (cizí klíč) na PARTIES pomocí pt_key a ptori_key.

Naším cílem je vypsat číslo účtu (acc_account_number) a majitele (pt_party_name).

Nyní si opět upravíme select. A naučíme se dva možné zápisy. První je specifický přímo pro Oracle SQL (tzv. Oracle syntaxe) a druhý je standardizovaný napříč různými modifikacemi SQL (tzv. ANSI syntaxe).

JOIN

Oracle

Syntaxe:

```
select
  tbl1.<atributy>,
  tbl2.<atributy>
from
  tabulka1 tbl1,
  tabulka2 tbl2
where
  tbl1.<join_atribut> = tbl2.<join_atribut> and
  tbl1.<podmínky> and
  tbl2.<podmínky>;
```

Poznámka:

Nemusíme mezi sebou spojovat pouze tabulky; namístě obou tabulek mohou figurovat poddotazy.

Můžete si všimnout, že za názvem tabulka1 je ještě tbl1. Tomuto se říká alias a slouží ke zjednodušení zápisu. Jinak bychom museli vypisovat vždy celý název tabulky, pokud by atribut neměl unikátní název napříč všemi použitými tabulkami. Ale ukažme si to na uvažovaném příkladu. Tentokrát klíčové slovo **as použít nejde!**

JOIN

Oracle

Příklad:

```
select
  acc.acc_account_number,
  pt.pt_party_name
from
  dwh_owner.accounts acc,
  dwh_owner.parties pt
where
  acc.pt_key = pt.pt_key and
  acc.ptori_key = pt.ptori_key and
  acc.acctp_key in (101, 202) and
  pt.ptori_key in (1, 2) and
  acc.acc_deleted_flag = 'N' and
  pt.pt_deleted_flag = 'N';
```

JOIN

ANSI

Syntaxe:

```
select
  tbl1.<atributy>,
  tbl2.<atributy>
from
  tabulka1 tbl1
join
  tabulka2 tbl2
on
  tbl1.<join_atribut> = tbl2.<join_atribut>
where
  tbl1.<podmínky> and
  tbl2.<podmínky>;
```

JOIN

ANSI

Příklad:

```
select
  acc.acc_account_number,
  pt.pt_party_name
from
  dwh_owner.accounts acc
join
  dwh_owner.parties pt
on
  acc.pt_key = pt.pt_key and
  acc.ptori_key = pt.ptori_key
where
  acc.acctp_key in (101, 202) and
  pt.ptori_key in (1, 2) and
  acc.acc_deleted_flag = 'N' and
  pt.pt_deleted_flag = 'N';
```

JOIN

Úkol:

Vypište počet nesmazaných klientů (= zaměstnanců) ze SAPu (ptori_key = 11) v závislosti na pohlaví. Použijte spojení na tabulku PERSON_GENDERS, nikoliv decode.

Řešení:

```
select
 g.psgen_desc,
 count(*) as pocet
from
 dwh_owner.parties p,
 dwh_owner.person_genders g
where
 p.ptori_key = 11 and
 p.pt_deleted_flag = 'N' and
 p.psgen_key = g.psgen_key
group by
 g.psgen_desc;
```

```
select
 g.psgen_desc,
 count(*) as pocet
from
 dwh_owner.parties p
join
 dwh_owner.person_genders g
on
 p.psgen_key = g.psgen_key
where
 p.ptori_key = 11 and
 p.pt_deleted_flag = 'N'
group by
 g.psgen_desc;
```

JOIN

Úkol:

Vypište počet nesmazaných klientů (= zaměstnanců) ze SAPu (ptori_key = 11) v závislosti na pohlaví a jejich organizační jednotce (org_unified_id). Použijte spojení ještě na tabulku ORGANIZATIONS.

Řešení:

```
select
 g.psgen_desc,
 o.org_unified_id,
 count(*) as pocet
from
 dwh_owner.parties p,
 dwh_owner.person_genders g,
 dwh_owner.organizations o
where
 p.ptori_key = 11 and
 p.pt_deleted_flag = 'N' and
 p.psgen_key = g.psgen_key and
 p.org_key = o.org_key
group by
 g.psgen_desc,
 o.org_unified_id;
```

```
select
 g.psgen_desc,
 o.org_unified_id,
 count(*) as pocet
from
 dwh_owner.parties p
join
 dwh_owner.person_genders g
on
 p.psgen_key = g.psgen_key
join
 dwh_owner.organizations o
on
 p.org_key = o.org_key
where
 p.ptori_key = 11 and
 p.pt_deleted_flag = 'N'
group by
 g.psgen_desc,
 o.org_unified_id;
```

Motivační příklad

Kolik nových klientů za Starbanku bylo založeno nebo zrušeno v lednu 2026 a jaký je přírůstek / úbytek v procentech oproti stavu k ultimo prosince 2025.

Co už umíme?

Vše potřebné!

Motivační příklad *(05_sql_novi.txt)*

```
select
 count(1) as novi_klienti
from
 (
  select
   pt_unified_key
  from
   dwh_owner.party_history
  where
   to_date('20260131', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
   ptori_key = 2 and
   pth_deleted_flag = 'N'
  minus
  select
   pt_unified_key
  from
   dwh_owner.party_history
  where
   to_date('20251231', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
   ptori_key = 2 and
   pth_deleted_flag = 'N'
 );
```

Motivační příklad *(06_sql_smazani.txt)*

```
select
 count(1) as smazani_klienti
from
 (
  select
   pt_unified_key
  from
   dwh_owner.party_history
  where
   to_date('20251231', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
   ptori_key = 2 and
   pth_deleted_flag = 'N'
  minus
  select
   pt_unified_key
  from
   dwh_owner.party_history
  where
   to_date('20260131', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
   ptori_key = 2 and
   pth_deleted_flag = 'N'
 );
```

Motivační příklad *(07_sql_vsichni.txt)*

```
select
 count(distinct pt_unified_key) as pocet_klientu
from
 dwh_owner.party_history
where
 to_date('20251231', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
 ptori_key = 2 and
 pth_deleted_flag = 'N';
```

Motivační příklad *(08_sql_starbank_01.txt)*



08_sql_starbank_01.txt

JOIN

V SQL rozlišujeme několik typů joinů, které můžeme použít.

Spojení, se kterým jsme se zatím setkali, se nazývá INNER JOIN. Zobrazí nám jen ta data, u kterých se spojení povede. Mohou však, ať už na jedné či druhé straně, existovat záznamy, které se nenapárují a z výsledku dotazu tak vypadnou. V některých situacích může být vhodné zobrazovat i tyto nenapojené záznamy.

Ukažme si to na následujícím příkladu.

Předpokládejme, že máme dvě tabulky. Jedna se jmenuje kancelář a je zde seznam všech místností. Druhá se nazývá zaměstnanec a u něj je uvedeno, ve které kanceláři sedí.

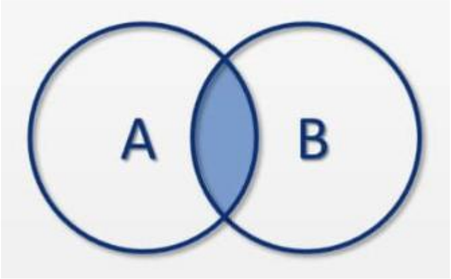
Mohou nastat 4 možné požadavky na výstup.

1) Vypište seznam lidí a kanceláři, kde sedí.
2) Vypište seznam místností a tam, kde je to možné, i všechny lidi, kteří tam sedí.
3) Vypište seznam lidí a tam, kde je to možné, i jejich kancelář.
4) Vypište seznam lidí a kanceláří, včetně prázdných místností i neusazených lidí.

JOIN

1) Vypište seznam lidí (B) a kanceláři (A), kde sedí.

S tímto typem jsme se již setkali, jmenuje se INNER JOIN a syntaxi známe.



Příklad

Úkol:

Vypište počty starbankových nesmazaných úvěrů (acctp_key=201) v závislosti na jejich stavu. Stav vypište textově; pomocí accst_key spojte accounts s account_status.

Řešení:

```
select
 count(*) as pocet,
 ast.accst_desc
from
 dwh_owner.accounts a,
 dwh_owner.account_status ast
where
 a.acctp_key = 201 and
 a.acc_deleted_flag = 'N' and
 a.accst_key = ast.accst_key
group by
 ast.accst_desc
order by
 pocet desc;
```

```
select
 count(*) as pocet,
 ast.accst_desc
from
 dwh_owner.accounts a
join
 dwh_owner.account_status ast
on
 a.accst_key = ast.accst_key
where
 a.acctp_key = 201 and
 a.acc_deleted_flag = 'N'
group by
 ast.accst_desc
order by
 pocet desc;
```

POCET	ACCST_DESC
3613569	UZATVORENÝ
773274	SCHVÁLENÝ
725841	UZATVORENÝ/STANDARDNÍ ZRUŠENÍ
269114	UZAVŘENÝ ODPISEM/ODPIS Z DŮVODU POSTOUPENÍ
135311	UZAVŘENÝ ODPISEM/DEFINITIVNĚ UZAVŘENÝ ODPIS
118781	UZAVŘENÝ ODPISEM/PŮVODNÍ TYP ODPISU
44873	NEREALIZOVANÝ PO SCHVÁLENÍ
40481	NEREALIZOVANÝ
37613	UZAVŘENÝ ODPISEM/ODPIS Z DŮVODU NEVYMOŽITELNOSTI
27822	ZESPLATNĚNÝ
6294	ZAMIETNUTÝ
2562	UZATVORENÝ/ZRUŠENÍ ÚVĚRU/ÚČTU TECHNICKÝM VYROVNÁNÍM
791	PRED SCHVÁLENÍM
50	UKONČENIE ZMLUVY
6	VÝPOVEĎ ZMLUVY

Příklad

Úkol:

Zkusme si obě tabulky omezit. V případě číselníku stavů pracujme jen s accst_key in (10, 11, 12, 13, 14) a v případě úvěrů jen s těmi, jejichž číslo začíná na 702187. Zkuste tabulky spojit a vypsat číslo účtu a stav.

Řešení:

```
select
 a.acc_account_number,
 a.accst_key as acc_accst_key,
 ast.accst_key,
 ast.accst_desc
from
 dwh_owner.accounts a,
 dwh_owner.account_status ast
where
 a.accst_key = ast.accst_key and
 a.acctp_key = 201 and
 a.acc_deleted_flag = 'N' and
 ast.accst_key in (10, 11, 12, 13, 14) and
 a.acc_account_number like '702187%'
order by
 a.acc_account_number;
```

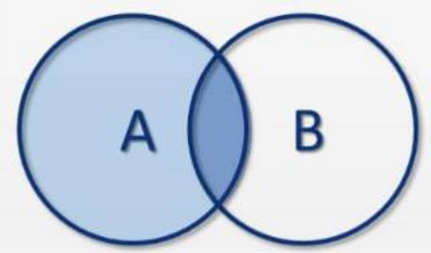
```
select
 a.acc_account_number,
 a.accst_key as acc_accst_key,
 ast.accst_key,
 ast.accst_desc
from
 dwh_owner.accounts a
join
 dwh_owner.account_status ast
on
 a.accst_key = ast.accst_key
where
 a.acctp_key = 201 and
 a.acc_deleted_flag = 'N' and
 ast.accst_key in (10, 11, 12, 13, 14) and
 a.acc_account_number like '702187%'
order by
 a.acc_account_number;
```

LEFT JOIN

2) Vypište seznam místností (A) a tam, kde je to možné, i všechny lidi (B), kteří tam sedí.

Případům 2), 3) a 4) se souhrnně říká OUTER JOIN a tomuto konkrétně LEFT (OUTER) JOIN.

Levý typ spojení tedy zachová všechny řádky tabulky A a tam, kde je to možné, doplní hodnoty z B.



LEFT JOIN

Oracle

Syntaxe:

```
select
  tbl1.<atributy>,
  tbl2.<atributy>
from
  tabulka1 tbl1,
  tabulka2 tbl2
where
  tbl1.<join_atribut> = tbl2.<join_atribut>(+) and
  tbl1.<podmínky> and
  tbl2.<podmínky>(+);
```

LEFT JOIN

Oracle

Příklad:

```
SELECT
  A.mistnost,
  nvl(B.jmeno, 'neobsazeno') as osazenstvo
FROM
  mistnosti A,
  lide B
WHERE
  A.id = B.mistnost_id(+);
```

LEFT JOIN

ANSI

Syntaxe:

```
select
  tbl1.<atributy>,
  tbl2.<atributy>
from
  tabulka1 tbl1
left (outer) join
  tabulka2 tbl2
on
  tbl1.<join_atribut> = tbl2.<join_atribut> and tbl2.<podmínky>;
where
  tbl1.<podmínky>;
```

Poznámka:

Klíčové slovo **outer** je nepovinné, můžete, ale nemusíte, ho psát. Pozor, pokud ve where klauzuli doplníte podmínku na tabulku tbl2, **bude fungovat jako filtr**.

LEFT JOIN

ANSI

Příklad:

```
SELECT
  A.mistnost,
  nvl(B.jmeno, 'neobsazeno') as osazenstvo
FROM
  mistnosti A
LEFT JOIN
  lide B
ON
  A.id = B.mistnost_id;
```

Příklad

Úkol:

Zkusme si obě tabulky omezit. V případě číselníku stavů pracujme jen s accst_key in (10, 11, 12, 13, 14) a v případě úvěrů jen s těmi, jejichž číslo začíná na 702187. Zkuste tabulky spojit pomocí left joinu a vypsat číslo účtu a stav.

Řešení:

```
select
 a.acc_account_number,
 a.accst_key as acc_accst_key,
 ast.accst_key,
 ast.accst_desc
from
 dwh_owner.accounts a,
 dwh_owner.account_status ast
where
 a.acctp_key = 201 and
 a.acc_deleted_flag = 'N' and
 a.acc_account_number like '702187%' and
 a.accst_key = ast.accst_key(+) and
 ast.accst_key(+) in (10, 11, 12, 13, 14)
order by
 a.acc_account_number;
```

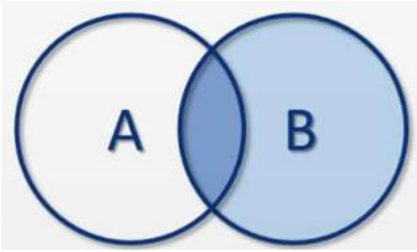
```
select
 a.acc_account_number,
 a.accst_key as acc_accst_key,
 ast.accst_key,
 ast.accst_desc
from
 dwh_owner.accounts a
left join
 dwh_owner.account_status ast
on
 a.accst_key = ast.accst_key and
 ast.accst_key in (10, 11, 12, 13, 14)
where
 a.acctp_key = 201 and
 a.acc_deleted_flag = 'N' and
 a.acc_account_number like '702187%'
order by
 a.acc_account_number;
```

RIGHT JOIN

3) Vypište seznam lidí (B) a tam, kde je to možné, i jejich kancelář (A).

Tento typ spojení je RIGHT (OUTER) JOIN.

Pravý typ spojení tedy zachová všechny řádky tabulky B a tam, kde je to možné, doplní hodnoty z A. Je to de facto levý typ spojení jen s přehozeným pořadím tabulek.



RIGHT JOIN

Oracle

Syntaxe:

```
select
  tbl1.<atributy>,
  tbl2.<atributy>
from
  tabulka1 tbl1,
  tabulka2 tbl2
where
  tbl1.<join_atribut>(+) = tbl2.<join_atribut> and
  tbl1.<podmínky>(+) and
  tbl2.<podmínky>;
```

RIGHT JOIN

Oracle

Příklad:

```
SELECT
  nvl(A.mistnost , 'neusazený') as mistnost
  B.jmeno
FROM
  mistnosti A,
  lide B
WHERE
  A.id(+) = B.mistnost_id;
```

RIGHT JOIN

ANSI

Syntaxe:

```
select
  tbl1.<atributy>,
  tbl2.<atributy>
from
  tabulka1 tbl1
right (outer) join
  tabulka2 tbl2
on
  tbl1.<join_atribut> = tbl2.<join_atribut> and tbl1.<podmínky>
where
  tbl2.<podmínky>;
```

RIGHT JOIN

ANSI

Příklad:

```
SELECT
  nvl(A.mistnost , 'neusazený') as mistnost
  B.jmeno
FROM
  mistnosti A
RIGHT JOIN
  lide B
ON
  A.id = B.mistnost_id;
```

RIGHT JOIN

Poznámka:

Nikdy v jednom dotazu nekombinujte LEFT a RIGHT join, stal by se tím zcela nečitelným!

Příklad

Úkol:

Zkusme si obě tabulky omezit. V případě číselníku stavů pracujme jen s accst_key in (10, 11, 12, 13, 14) a v případě úvěrů jen s těmi, jejichž číslo začíná na 702187. Zkuste tabulky spojit pomocí right joinu a vypsat číslo účtu a stav.

Řešení:

```
select
 a.acc_account_number,
 a.accst_key as acc_accst_key,
 ast.accst_key,
 ast.accst_desc
from
 dwh_owner.accounts a,
 dwh_owner.account_status ast
where
 ast.accst_key in (10, 11, 12, 13, 14) and
 a.accst_key(+) = ast.accst_key and
 a.acctp_key(+) = 201 and
 a.acc_deleted_flag(+) = 'N' and
 a.acc_account_number(+) like '702187%'
order by
 a.acc_account_number;
```

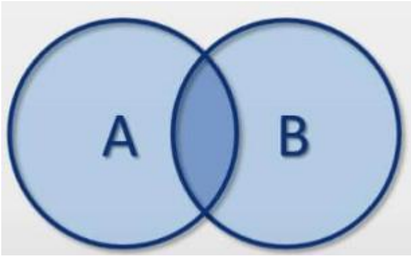
```
select
 a.acc_account_number,
 a.accst_key as acc_accst_key,
 ast.accst_key,
 ast.accst_desc
from
 dwh_owner.accounts a
right join
 dwh_owner.account_status ast
on
 a.accst_key = ast.accst_key and
 a.acctp_key = 201 and
 a.acc_deleted_flag = 'N' and
 a.acc_account_number like '702187%'
where
 ast.accst_key in (10, 11, 12, 13, 14)
order by
 a.acc_account_number;
```

FULL JOIN

4) Vypište seznam lidí a kanceláří, včetně prázdných místností i neusazených lidí.

Tento typ spojení je FULL (OUTER) JOIN.

Full outer join zachová všechny řádky tabulek A i B a tam, kde je to možné, hodnoty spojí.



FULL JOIN

Oracle

Syntaxe:

Žádná, je nutné použít ANSI syntaxi.

FULL JOIN

ANSI

Syntaxe:

```
select
  tbl1.<atributy>,
  tbl2.<atributy>
from
  (
    select
      <atributy>
    from
      tabulka1
    where
     <podmínky>
) tbl1
full (outer) join
  (
    select
      <atributy>
    from
      tabulka2
    where
     <podmínky>
) tbl2 on
  tbl1.<join_atribut> = tbl2.<join_atribut>;
```

FULL JOIN

ANSI

Příklad:

```
SELECT
  nvl(A.mistnost , 'neusazený') as mistnost
  nvl(B.jmeno, 'neobsazeno') as osazenstvo
FROM
  mistnosti A
FULL JOIN
  lide B
ON
  A.id = B.mistnost_id;
```

Příklad

Úkol:

Zkusme si obě tabulky omezit. V případě číselníku stavů pracujme jen s accst_key in (10, 11, 12, 13, 14) a v případě úvěrů jen s těmi, jejichž číslo začíná na 702187. Zkuste tabulky spojit pomocí full joinu a vypsat číslo účtu a stav.

Řešení:

```
select
  a.acc_account_number,
  a.accst_key as acc_accst_key,
  ast.accst_key,
  ast.accst_desc
from
  (
    select
      acc_account_number,
      accst_key
    from
      dwh_owner.accounts
    where
      acctp_key = 201 and
      acc_deleted_flag = 'N' and
      acc_account_number like '702187%'
  ) a
full join
  (
    select
      accst_desc,
      accst_key
    from
      dwh_owner.account_status ast
    where
      accst_key in (10, 11, 12, 13, 14)
  ) ast
on
  a.accst_key = ast.accst_key
order by
  a.acc_account_number;
```

JOIN

Pro zajímavost existují ještě další typy joinů.

1) **NATURAL JOIN** – provede spojení dvou tabulek podle všech sloupců, které se jmenují stejně. Může být inner i outer.

2) **CROSS JOIN** – provede kartézský součin, tedy spojí každý řádek s každým. V Oracle syntaxi toho dosáhneme vypuštěním join podmínek.

EXISTS

Nyní se podíváme v detailu i na poslední operátor, který jsme na začátku zmínili. A to operátor **EXISTS**.

Syntaxe:

```
select
 *
from
 tabulka1 tbl1
where
 (not) exists
  (
   select
    1
   from
    tabulka2 tbl2
   where
    tbl2.<join_podmínka> = tbl1.<join_podmínka>
  );
```

Tento dotaz vrátí všechny řádky, pro které (ne)existuje alespoň 1 řádek v poddotazu. Je zcela jedno, co tento poddotaz vrací za hodnoty.

EXISTS

Příklad:

```
select
 *
from
 dwh_owner.person_genders g
where
 exists
  (
   select
    1
   from
    dwh_owner.parties pt
   where
    pt.ptori_key = 32 and
    pt.psgen_key = g.psgen_key
  );
```

PSGEN_KEY	PSGEN_SOURCE_SYSTEM_ID	PSGEN_SOURCE_ID	PSGEN_DESC	PSGEN_UNIFIED_ID	PSGEN_UNIFIED_DESC
-1	XNA	XNA	N/A	XNA	N/A

EXISTS

Úkol:

Vypište pomocí příkazu EXISTS ty nesmazané organizační jednotky se systémem HR0, které se nevyskytují u žádného klienta ze SAPu.

Řešení:

```
select
 o.org_unified_id
from
 dwh_owner.organizations o
where
 o.org_deleted_flag = 'N' and
 o.org_source_system_id = 'HR0' and
 not exists
  (
   select
    1
   from
    dwh_owner.parties p
   where
    p.ptori_key = 11 and
    p.org_key = o.org_key
  );
```

WITH

Na chvíli se vraťme k našemu motivačnímu příkladu. Mohli jste si všimnout, že na některá data jsme sahali opakovaně. Data za prosinec jsme vybírali třikrát, za leden dvakrát. Oracle nabízí možnost, abychom tento duplicitní kód mohli napsat jen jednou. K tomu slouží klauzule **WITH**.



08_sql_starbank_01.txt

WITH

Syntaxe:

```
with
 sel
as
 (
  select
   <sloupce>
  ...
 ),
 sel2
as
 (
  ...
 )...
select
 <sloupce>
from
 sel;
```

Klauzule WITH definuje poddotazy ještě před samotným příkazem select.

Motivační příklad

V našem případě bychom mohli definovat data za prosinec a leden, např. takto:

```
with
  prosinec
as
  (
    select
      pt_unified_key
    from
      dwh_owner.party_history
    where
      to_date('20251231', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
      ptori_key = 2 and
      pth_deleted_flag = 'N'
  ),
  leden
as
  (
    select
      pt_unified_key
    from
      dwh_owner.party_history
    where
      to_date('20260131', 'YYYYMMDD') between pth_valid_from and pth_valid_to and
      ptori_key = 2 and
      pth_deleted_flag = 'N'
  )
```

Motivační příklad

A poté bychom mohli vybrat nové klienty níže uvedeným dotazem:

```
select
 count(1) as pocet
from
 (
  select
   pt_unified_key
  from
   leden
  minus
  select
   pt_unified_key
  from
   prosinec
 );
```

Motivační příklad *(09_sql_starbank_02.txt)*

Upravený dotaz tak může vypadat takto:



09_sql_starbank_02.txt

Příklad

Úkol:

Vypište (na jeden řádek!) čísla účtů všech nesmazaných běžných účtů (acctp_key in (101, 202, 602)) spolu s počtem kreditních transakcí za říjen, listopad a prosinec 2025. V případě, že záznam o počtu neexistuje , použijte nulu.

Počty transakcí najdete v dwh_owner.CURRENT_ACCOUNT_FACTS v ACCFC_MTD_CR_TRANS_CNT. Období je ACCFC_PERIOD. Zkuste využít kombinaci CASE a MAX (či jiné agregační funkce).

Řešení:

```
select
  a.acc_account_number,
  nvl(trn.prosinec, 0) as prosinec,
  nvl(trn. listopad, 0) as listopad,
  nvl(trn.rijen, 0) as rijen
from
  dwh_owner.accounts a,
  (
    select
      acc_key,
      acctp_key,
      max(case when accfc_period = to_date('20251231', 'YYYYMMDD') then accfc_mtd_cr_trans_cnt else 0 end) as prosinec,
      max(case when accfc_period = to_date('20251130', 'YYYYMMDD') then accfc_mtd_cr_trans_cnt else 0 end) as listopad,
      max(case when accfc_period = to_date('20251031', 'YYYYMMDD') then accfc_mtd_cr_trans_cnt else 0 end) as rijen
    from
      dwh_owner.current_account_facts
    where
      accfc_period between to_date('20251031', 'YYYYMMDD') and to_date('20251231', 'YYYYMMDD') and
      acctp_key in (101, 202, 602)
    group by
      acc_key,
      acctp_key
  ) trn
where
  a.acctp_key in (101, 202, 602) and
  a.acc_deleted_flag = 'N' and
  a.acc_key = trn.acc_key(+) and
  a.acctp_key = trn.acctp_key(+)
order by
  a.acc_account_number;
```

ACC_ACCOUNT_NUMBER	PROSINEC	LISTOPAD	RIJEN
0	0	0	0
0000001193	2	2	3
0000001329	0	0	0
0000002073	0	0	0
0000002233	0	0	0
0000002399	52	33	40
0000002399	24	26	21
0000003113	1	1	0
0000003404	0	0	0
0000004183	1	1	1
0000004263	0	0	0
0000004319	0	0	0
0000004474	0	0	0
0000005223	0	0	0
0000005389	0	0	0
0000005434	0	0	0
0000006023	2	2	4
0000006103	1	1	1

Malá ochutnávka *(10_sql_connect.txt)*

Vzpomínáte, na ukázku poddotazů, kde jsme měli za úkol vypsat první tři dny v roce? Představme si podobnou úlohu, vypište v řádcích všechny dny v daném měsíci. Pokud bychom věděli, kolik má dní, mohli bychom použít přesně tolik „zunionovaných“ selectů. Kdyby ne, už by byla úloha výrazně těžší. V Oraclu existuje elegantní řešení, jednoduchý dotaz

```
select
  level
from
  dual
connect by
  level <= <počet>;
```

Výpis všech dní v aktuální měsíci by pak mohl vypadat takto:

```
select
  trunc(sysdate, 'MM') + level - 1 as datum
from
  dual
connect by
  level <= (last_day(trunc(sysdate)) - trunc(sysdate, 'MM') + 1);
```

Co znamená **level**, **connect by** a další informace k tzv. hierarchickým dotazům se dozvíme příště.

Dotazy

Máte nějaké dotazy?

Děkuji za pozornost a příště naviděnou!